Final Submission

Image Processing/ CSE381

Done by:
Mariam Tarek 23P0214

Mariam Ibrahim 23P0123

Omar Abouraia 23P0100

Hossam Osama 23P0010

Lujayn Mohamed 23P0187

Supervised by:
Prof. Mahmoud Khalil

Gravity Falls Puzzle Solver – Project Report (Milestone 2)

1. Introduction

This project aims to automatically solve jigsaw puzzles from images, focusing on both 2x2, 4x4 puzzles and 8x8 puzzles. The system uses classical computer vision techniques, without any machine learning or deep learning, to segment puzzle pieces, compute edge similarity, and assemble the puzzle. The main goal is to emulate the human process of solving puzzles by analyzing piece contours and edge features.

---

2. Image Processing Pipeline

The pipeline is divided into four main stages:

2.1 Preprocessing

- Objective: Prepare images for segmentation and matching.
- Steps:
  - Images are loaded using OpenCV.
  - For 2x2 puzzles, the image is divided into four equal regions using the segment_exact_2x2 function.
  - For 4x4 puzzles, images are divided into 16 equal pieces using uniform cropping (h_step and w_step).
- Justification:
  - Simple grid-based segmentation is sufficient as input images have well-defined puzzle layouts.
  - Ensures consistency for subsequent edge comparison.

2.2 Edge Representation

- Objective: Represent puzzle piece edges in a way that allows comparison.
- Techniques:
  - Color conversion to LAB space: Edges are converted from BGR to LAB color space.
    - This improves perceptual uniformity, as LAB separates luminance from chromaticity.

    - Edge cropping: Only the outer 2–5 pixels of each piece are considered to focus on the edge features.
    - Variance weighting: Low-variance (flat) edges are penalized during 4x4 matching.
- Justification:
    - LAB color space provides better color difference measurement than RGB.
    - Focusing on edge pixels reduces noise from inner textures.
    - Penalizing flat edges avoids incorrect matches in uniform areas.

2.3 Edge Matching

- 2x2 Puzzles:
    - Simple matching (Pass 1): Computes the mean absolute difference between adjacent edges.
    - Strong matching (Pass 2): Adds a flat-edge penalty and considers outer border penalties to improve matching robustness.
- 4x4 Puzzles:
    - Uses a Beam Search algorithm:
        - Maintains a set of the best candidate placements (beam width = 100).
        - At each step, adds a new piece to open slots, computing the cost based on adjacent edge similarity.
        - Deduplicates states to prevent redundant calculations.
- Justification:
    - Two-pass solver ensures 2x2 puzzles are solved efficiently.
    - Beam search for 4x4 handles the combinatorial explosion of possible arrangements efficiently.

2.4 Stitching & Grading

- Objective: Reconstruct the full puzzle and evaluate the correctness.

- Steps:
  - Pieces are stitched using numpy.hstack and numpy.vstack to form the full image.
  - Optional trimming of edges (CLEAN_TRIM_SIZE) removes borders that may affect grading.
  - The reconstructed image is compared to ground truth using Structural Similarity Index (SSIM), including slight pixel shifts to account for alignment errors.
- Justification:
  - SSIM measures perceptual similarity, better capturing human-perceived correctness than raw pixel difference.

---

3. Folder Organization and Output

- 2x2 puzzles: Outputs are saved in subfolders:
  - correct_pass1
  - correct_pass2
  - fail
- 4x4 puzzles: Outputs are saved in subfolders:
  - correct
  - fail
- Justification:
  - Organizing outputs by status allows quick assessment of algorithm performance.

---

4. Key Decisions and Design Choices

- LAB color space for better color comparison.
- Two-pass solver for 2x2 to balance speed and robustness.
- Beam search for 4x4 to handle combinatorial complexity.

- Edge trimming and variance penalty to avoid misalignment from flat or noisy edges.
- SSIM for grading, with pixel shifts to accommodate small misalignments.

---

6. Limitations and Challenges

While the system performs well on small puzzles (2x2 and 4x4), it **fails to reconstruct larger puzzles such as 8x8**. The primary reason is the **combinatorial explosion** of possible piece arrangements, which overwhelms the current beam search algorithm. Even with a high beam width, the algorithm could not find correct placements, and the **accuracy dropped to 0%** for 8x8 puzzles.

This limitation highlights the need for more advanced optimization strategies or heuristics to handle larger puzzles effectively, such as:

- More sophisticated pruning methods for the search space.
- Hierarchical or piece-clustering approaches before full assembly.
- Integration of additional features for edge matching, such as texture or corner shapes.

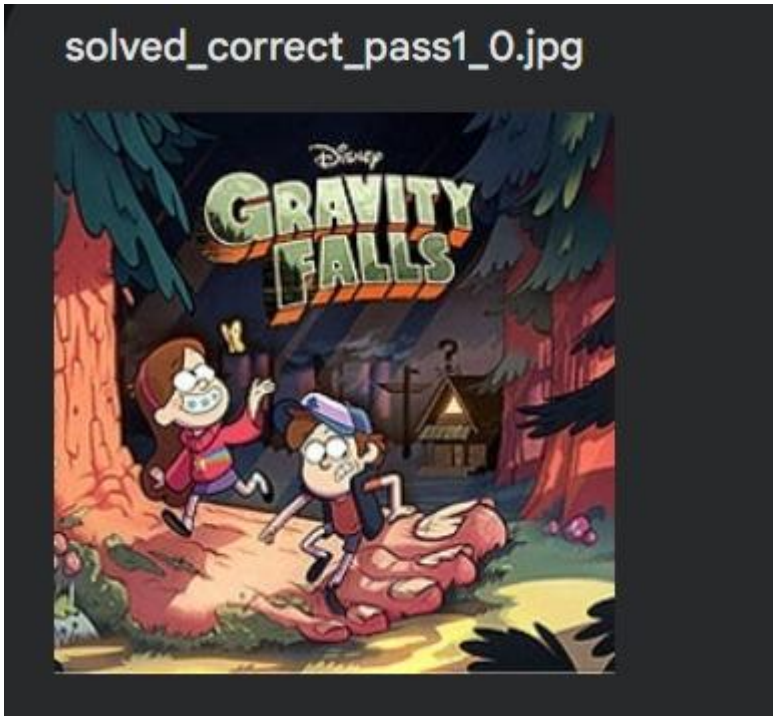
Despite this limitation, the current system successfully demonstrates the classical computer vision pipeline for small-scale puzzle reconstruction.

---

6. Output and Results

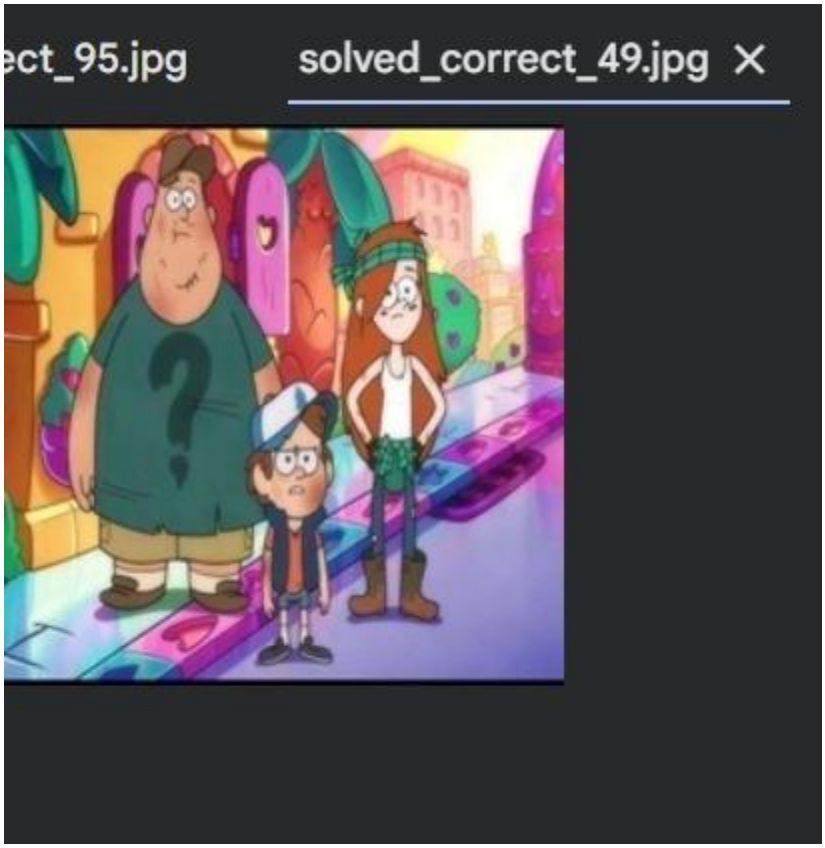
The system produces visual outputs for both **2x2** and **4x4** puzzles, organized in subfolders according to their reconstruction status, here's the after output visualization:

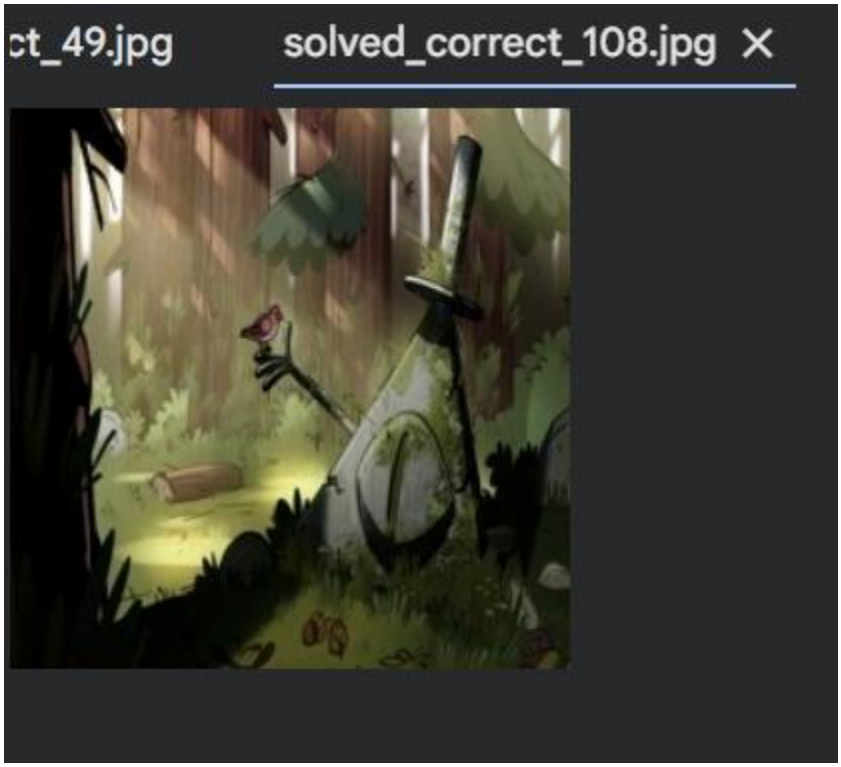
2x2 puzzle:


solved_correct_pass1_0.jpg
Disney
GRAVITY FALLS

solved_correct_pass1_103.jpg

4x4 puzzle:


ect_95.jpg
solved_correct_49.jpg


ct_49.jpg
solved_correct_108.jpg

7. Conclusion

The implemented system successfully segments, analyzes, and reconstructs 2x2 and 4x4 puzzles using purely classical computer vision techniques. The design choices, including color-space conversion, edge trimming, two-pass matching, and beam search, ensure robustness against varying puzzle orientations and minor noise.